Laboratory Report: The Arduino Nano 33 BLE and ROS2 Serial

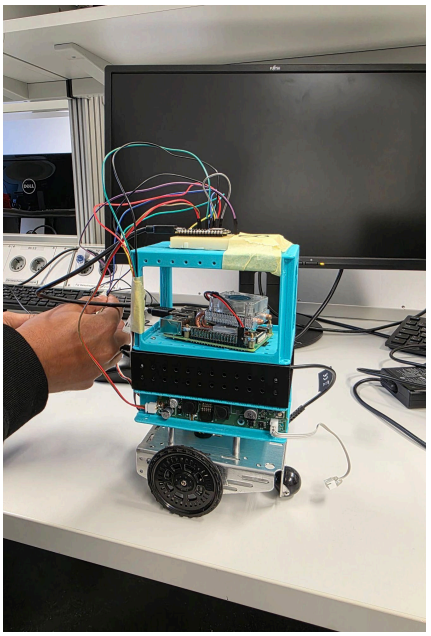
Course: Advanced Robotics, TH Köln, Prof. Elena Algorri
Team: Abhishek Chandorkar, Chinmay Vaity
Date: 26-10-2026

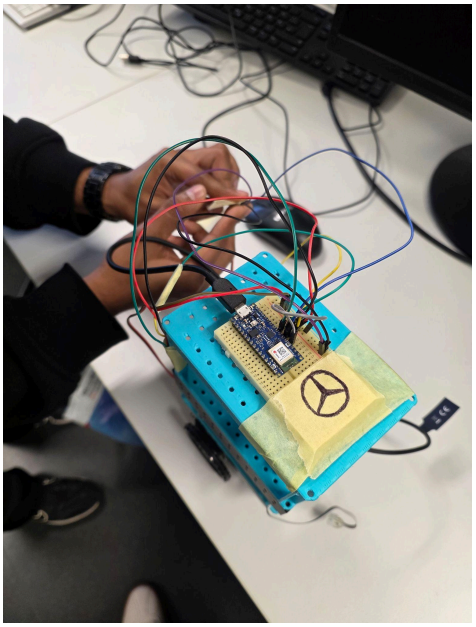
---

1. Building the Robot

All mechanical parts (chassis, servos, wheels, mounting plates and boxes) were part of the robotics kit. The robot is built as a stack of layers on the aluminium chassis. From bottom to top:

1. **Chassis:** the aluminium chassis came with both continuous-rotation servos and their wheels already mounted, plus a ball caster at the front as the third support point. The LiPo battery sits in the slot provided in the chassis.
2. **Voltage regulator layer:** a blue mounting plate on top of the chassis carries the DC-DC voltage regulator board, which converts the LiPo voltage to 6 V for the servos.
3. **Black box (power bank):** the power bank that supplies the Raspberry Pi is placed inside the black box above the regulator. We deliberately put it in the middle of the stack. As the heaviest part apart from the LiPo, placing it on top would have raised the centre of gravity and could have tipped the robot backward.
4. **Blue frame (Raspberry Pi):** an open blue frame is screwed on top of the black box. The Raspberry Pi 4, fitted with an active cooler (fan and heat-pipe heatsink), is mounted inside it. The open sides keep the USB ports reachable and let air reach the cooler.
5. **Breadboard (Arduino Nano):** on top of the blue frame is a small breadboard holding the Arduino Nano 33 BLE and all signal and power connections. It is fixed with tape, and the jumper wires are taped along the frame so they don't hang loose.





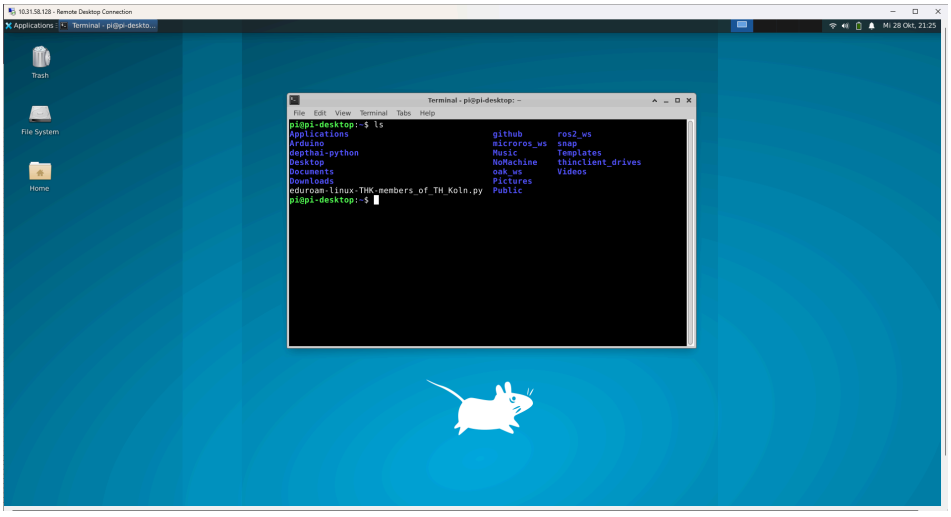
- **Raspberry Pi:** powered by the power bank over USB-C.
- **Arduino Nano 33 BLE:** connected to the Raspberry Pi by a USB cable, which provides both power and the serial link (/dev/ttyACM0).
- **Servos:** powered with 6 V from the regulator's output. The positive output (red wire) goes only to the servo power wires and never touches any Nano pin.
- **Common ground:** the regulator's negative output (brown wire), the servo grounds and a GND pin of the Nano are all connected on the breadboard's ground rail.
- **Servo cables:** they end in female connectors and are connected to the breadboard with male-to-male jumper wires.
- **LiPo:** connected to the regulator's input, and switched on with the regulator's push button. The LED shows it is running.
- **Arduino Nano to Servo:**

Signal	Nano Pin
Left servo, signal	D3
Right servo, signal	D5
Left servo, feedback	D2
Right servo, feedback	D4

Remote connection to the Raspberry Pi

To run the Pi without a monitor, keyboard and mouse, we set up two kinds of remote access:

- **xrdp (remote desktop):** we installed xrdp on the Pi and connected from the laptop with Windows *Remote Desktop Connection*. This gives the full graphical desktop of the Pi, which we used for the Arduino IDE.



- **VS Code Remote-SSH:** we used the *Remote - SSH* extension (F1 → *Remote-SSH: Connect to Host* → ADC@<Pi-IP>) to edit files in ~/ros2_ws and to run ROS2 in terminals directly on the Pi.

Laptop and Pi were connected through mobile phone hotspot..

2. Code Changes

2.1 Arduino sketch

Per-servo, per-direction calibration. The original sketch used the same stop value (94) and offset (±6) for both servos. We replaced these with separate stop values for each servo and separate speed offsets for each direction:

Servo	Stop value	Forward offset	Backward offset
Left	94	6	12
Right	93	9	7

The servos are mounted mirrored, so forward is stop + offset for the left servo and stop − offset for the right one. We found the stop values with a separate calibration sketch, which sets each servo's value from the Serial Monitor (e.g. R93, L94).

Movement as a function. The repeated attach/write/delay/detach code in each case was moved into one function, drive (left, right, duration).

New command g (180° spin). The left wheel turns forward and the right wheel backward for spinTime milliseconds, so the robot turns on the spot. We tuned spinTime experimentally to 1010 ms for one full 180° turn.

Command protocol: single characters f, b, s, g, at 9600 baud. Each command moves the robot for 500 ms (g: spinTime). All other characters, including the newline sent by the ROS2 node, are ignored.

2.2 ROS2 serial node (serial_node_ACM0.py)

The node subscribes to /cvaity_serial_ACM0 (std_msgs/String) and writes each command to /dev/ttyACM0 at 9600 baud. We made these changes to the given code:

- **Own topic name:** algorri_serial_ACM0 → cvaity_serial_ACM0.

3. Problems and Solutions

3.1 Servos not responding correctly (missing common ground)

Problem: at first the left servo did not move at all, and the right servo only moved while we pressed on the jumper connections. Later, both servos spun continuously in one direction, whatever command we sent.
Cause: the Nano's GND was not connected to the regulator's ground. A servo reads its control signal as a voltage relative to its own ground, so without a common reference the PWM signal from the Nano was meaningless to the servos. Pressing on the connections occasionally created a brief, unintended ground contact, which is why the servos sometimes reacted.
Solution: we connected a GND pin of the Nano to the shared ground rail. Both servos then responded reliably to every command. Only the positive supply is kept separate, not the ground.

3.2 Unequal wheel speeds

Problem: with the original values, the right wheel was faster on f, the left was faster on b, and the right wheel kept creeping on s.
Cause: the neutral point of low-cost continuous servos differs from unit to unit, and their response is not the same in both directions.
Solution: calibration with separate stop values and direction-dependent offsets (Section 2.1).